Cost-Efficient Foreground Segmentation Using a Lightweight Depthwise-Separable U-Net under a Single-Core CPU Budget

Rahul Kumar Bhikundia
Roll Number: M26CSA006
CSL7360: Computer Vision — Assignment 2
Indian Institute of Technology Jodhpur

Abstract

This report presents a technical overview and experimental evaluation of a cost-efficient binary foreground segmentation pipeline designed to segment the primary object of interest within a 256×256 image under a strict single-core CPU latency budget ($\leq$ 50 ms per image) and compact storage footprint. Following Track B, a lightweight U-Net-inspired fully convolutional neural network (`SmallUNet`) was designed and trained completely from scratch on the DUTS-TR dataset without relying on pretrained weights. The architecture combines depthwise-separable convolutions, batch normalization, bilinear upsampling, skip connections, and aggressively pruned channel widths ($3 \to 8 \to 16 \to 32 \to 48$) operating directly at 256×256 resolution, resulting in an ultra-compact model with $15,949$ trainable parameters occupying $129,326$ bytes (≈ 0.1233 MiB, or 3.08% of the 4 MiB reference budget) on disk. Evaluated across the 5,019-image DUTS-TE test set, the pipeline achieves a mean Intersection over Union (mIoU) of 0.64133, a Boundary F1-score (BF1) of 0.30448, and a quality score of $Q = 0.54027$ at an average CPU inference time of 25.2291 ms, yielding a final Cost-Efficient Segmentation (CES) score of 0.48408.

1 Objective

The objective of this work is to design and train from scratch a computationally efficient binary foreground segmentation pipeline that assigns a binary class label (1 for main object, 0 for background) to every pixel of an input photograph while operating within a strict single-threaded CPU compute budget. Specifically, the target pipeline must satisfy the following requirements:

- Achieve an average single-core CPU inference latency of at most 50 ms per photograph across the 5,019-image DUTS-TE test benchmark.
- Train a small Fully Convolutional Network (FCN) completely from scratch using only primitive PyTorch layers without pretrained weights or external segmentation libraries.
- Maximize the Cost-Efficient Segmentation (CES) metric by balancing region overlap (mIoU), contour accuracy (Boundary F1), single-threaded CPU runtime (T_{CPU}), and parameter file size (S_{disk}).
- Generalize across challenging real-world campus and hostel scenes exhibiting indoor and outdoor lighting, scale variations, cluttered backgrounds, and partial occlusions.

2 Overall Pipeline Architecture

The segmentation pipeline is divided into offline image resizing, timed BGR-to-RGB colour conversion and tensor normalization, three-stage depthwise-separable U-Net encoding ($8 \to 16 \to$

32 channels), latent bottleneck processing (48 channels at 32×32 resolution), three-stage bilinear decoding with concatenation skip connections ($80 \to 32$, $48 \to 16$, and $24 \to 8$ channels), 1×1 pointwise logit projection, and direct zero-logit thresholding. Figure 1 illustrates the complete execution dataflow, highlighting the operations included within the timed CPU benchmark.

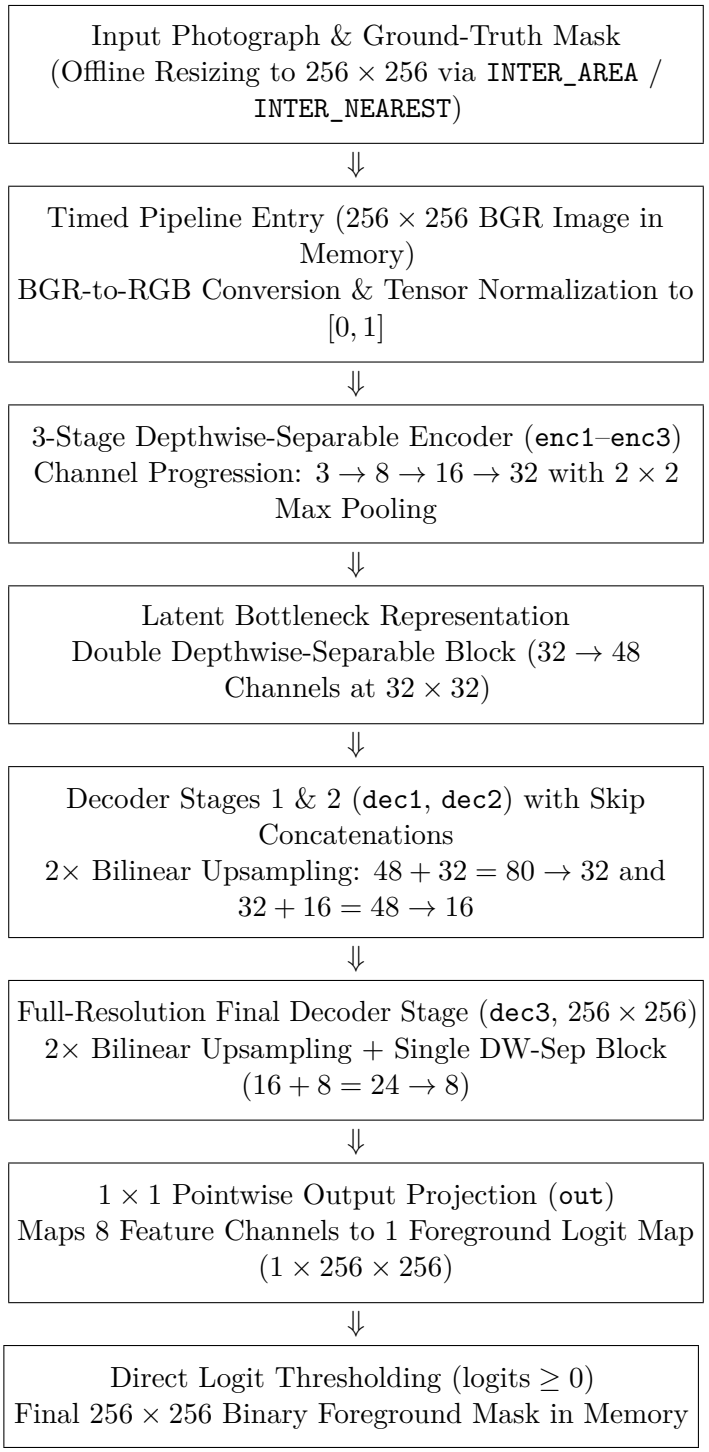


Figure 1: System architecture and timed execution dataflow of the `SmallUNet` segmentation pipeline.

3 Dataset and Preprocessing

3.1 DUTS Dataset Partitioning

The DUTS salient object detection dataset was utilized for model training, validation, and official benchmarking. To ensure strict separation between model selection and final evaluation, the 10,553-image `DUTS-TR` dataset was partitioned into a 90% training subset and a 10% validation subset using a fixed random seed (`torch.Generator().manual_seed(42)`). The official `DUTS-TE` test dataset was reserved exclusively for final evaluation after freezing the selected model checkpoint:

- **DUTS-TR Training Subset:** $9,497$ image–mask pairs, Tensor shapes: $(3, 256, 256)$ and $(1, 256, 256)$
- **DUTS-TR Validation Subset:** $1,056$ image–mask pairs, used for hyperparameter tuning and checkpoint selection
- **DUTS-TE Official Test Set:** $5,019$ image–mask pairs, reserved strictly for final testing and CPU benchmarking

3.2 Image and Mask Resizing Protocol

All images and ground-truth segmentation masks are standardized to a spatial resolution of 256×256 pixels according to Section 1b of the specification:

1. Input photographs are read using OpenCV and resized to 256×256 using pixel-area interpolation (`cv2.INTER_AREA`).
2. Ground-truth grayscale masks are resized to 256×256 using nearest-neighbor interpolation (`cv2.INTER_NEAREST`) to preserve discrete boundary labels without introducing intermediate gray values.
3. Mask pixels satisfying intensity ≥ 128 are assigned to the foreground object class (1.0), and pixels < 128 are assigned to the background class (0.0).
4. Input images are converted from BGR to RGB format, permuted to channel-first tensor format $(3, 256, 256)$, and normalized to the range $[0, 1]$ by dividing by 255.0.

3.3 Personal Photograph Dataset (`captured/`)

To evaluate out-of-distribution generalization on real-world scenes, six photographs were captured in campus and hostel environments covering indoor and outdoor scenes, daylight and tube lighting, near and far objects, cluttered backgrounds, and partial occlusion. Via the preparation script (`prepare_captured.py`), the dataset was structured into three subdirectories (`original/`, `images_256/`, and `masks_256/` inside `captured/`) with matching base filenames (`photo1` through `photo6`). The six scenes depict a potted plant (`photo1`), a matte black water bottle on a cluttered desk (`photo2`), a sneaker on an indoor floor (`photo3`), a partially occluded biscuit packet behind an electronic controller/monitor unit (`photo4`), a distant outdoor water dispenser (`photo5`), and a bicycle parked against a brick wall (`photo6`).

4 Proposed Network Architecture (`SmallUNet`)

To satisfy the 50 ms single-core CPU inference budget while operating directly at full 256×256 resolution to preserve small-object boundaries, a lightweight U-Net-inspired encoder–decoder architecture (`SmallUNet`) was implemented from scratch in PyTorch. Rather than relying

on standard 3×3 convolutions, which require $9\,C_{\text{in}}C_{\text{out}}HW$ multiply-accumulate operations (MACs), each convolutional stage factorizes spatial filtering and channel mixing into a **depthwise-separable convolution**:

1. **Depthwise Convolution:** Applies a 3×3 spatial filter independently to each input channel (padding $= 1$, groups $= C_{\text{in}}$), followed by BatchNorm2d(C_{in}) and in-place ReLU activation.

2. **Pointwise Convolution:** Applies a 1×1 convolution across channels ($C_{\text{in}} \to C_{\text{out}}$), followed by BatchNorm2d(C_{out}) and in-place ReLU activation.

Each standard encoder and decoder block (`ConvBlock`) stacks two consecutive depthwise-separable units, whereas the highest-resolution final decoder stage (`FinalDecoderBlock`) uses a single depthwise-separable unit ($24 \to 8$ channels at 256×256) followed by a 1×1 output convolution ($8 \to 1$ channel) to minimize full-resolution compute overhead. By bounding channel widths at $\{8, 16, 32, 48\}$, the entire network contains only $15,949$ trainable parameters ($129,326$ bytes on disk). Table 1 summarizes the layer-by-layer architecture and parameter count.

Table 1: Layer-by-layer specification and trainable parameter breakdown of `SmallUNet`.

Stage	**Block / Operation**	**Channels ($C_{\text{in}} \to C_{\text{out}}$)**	**Spatial Resolution**	**Parameters**
`enc1 + pool1`	`ConvBlock` (2× DW-Sep) + 2×2 MaxPool	$3 \to 8$	$256 \times 256 \to 128 \times 128$	268
`enc2 + pool2`	`ConvBlock` (2× DW-Sep) + 2×2 MaxPool	$8 \to 16$	$128 \times 128 \to 64 \times 64$	768
`enc3 + pool3`	`ConvBlock` (2× DW-Sep) + 2×2 MaxPool	$16 \to 32$	$64 \times 64 \to 32 \times 32$	2,304
`bottleneck`	`ConvBlock` (2× DW-Sep)	$32 \to 48$	32×32	5,088
`up1 + dec1`	2× Bilinear + Concat(`enc3`) + `ConvBlock`	$48 + 32 = 80 \to 32$	$32 \times 32 \to 64 \times 64$	5,120
`up2 + dec2`	2× Bilinear + Concat(`enc2`) + `ConvBlock`	$32 + 16 = 48 \to 16$	$64 \times 64 \to 128 \times 128$	1,888
`up3 + dec3`	2× Bilinear + Concat(`enc1`) + `FinalDecoderBlock`	$16 + 8 = 24 \to 8$	$128 \times 128 \to 256 \times 256$	504
`out`	1×1 Pointwise Output Convolution	$8 \to 1$	256×256	9
Total	**Complete Trainable Network**	—	$\mathbf{256 \times 256 \to 256 \times 256}$	$\mathbf{15,949}$

5 Training Configuration and Checkpoint Selection

The network was trained from scratch on the 9,497-image DUTS-TR training split using an NVIDIA GTX 1650 GPU. Optimization was performed over 50 epochs (with early stopping patience set to 7 epochs) using a batch size of 16, the Adam optimizer (lr $= 0.001$), and pixel-wise Binary Cross-Entropy with Logits Loss (`nn.BCEWithLogitsLoss()`):

$$\mathcal{L}_{\text{BCE}}(\hat{z}, y) = -\frac{1}{N}\sum_{i=1}^{N}\Big[y_i \log \sigma(\hat{z}_i) + (1 - y_i)\log\big(1 - \sigma(\hat{z}_i)\big)\Big] \tag{1}$$

where $\hat{z}_i$ denotes the predicted logit at pixel i, $y_i \in \{0, 1\}$ denotes the ground-truth label, and $\sigma(\hat{z}_i) = (1 + e^{-\hat{z}_i})^{-1}$. During inference, foreground classification is performed via direct logit thresholding ($\hat{z}_i \geq 0$), which is equivalent to $\sigma(\hat{z}_i) \geq 0.5$ and avoids computing the sigmoid function across all $65,536$ pixels.

Validation loss was evaluated after every epoch on the 1,056-image validation set, decreasing from 0.4087 in Epoch 1 (training loss 0.4617) to a minimum validation loss of 0.2603 in Epoch 44 (training loss 0.2500). The checkpoint achieving the lowest validation loss was saved to `checkpoints/best_model_4.pth` ($129,326$ bytes) and frozen for final evaluation. Table 2 summarizes the training configuration.

Table 2: Summary of training hyperparameters and checkpoint selection.

Configuration Parameter	**Value / Specification**
Framework	PyTorch (Track B FCN from scratch)
Training / Validation Split	$9,497$ Training / $1,056$ Validation (Seed = 42)
Input / Operating Resolution	256×256 Full-Resolution Forward Pass
Batch Size / Max Epochs	16 / 50 Epochs (Early Stopping Patience = 7)
Optimizer / Learning Rate	Adam / 1.0×10^{-3}
Loss Function	Binary Cross-Entropy with Logits (`BCEWithLogitsLoss`)
Validation Loss Trajectory	0.4087 (Epoch 1) $\rightarrow$ 0.2603 (Epoch 44, Best)
Training Hardware	NVIDIA GeForce GTX 1650 GPU
Saved Parameter File	`checkpoints/best_model_4.pth` ($129,326$ bytes)

6 Evaluation Metrics

6.1 Mean Intersection over Union (mIoU)

To evaluate region-level overlap, true positives (TP), false positives (FP), false negatives (FN), and true negatives (TN) are accumulated into a single global 2×2 confusion matrix across all images in an evaluation set. The class-wise IoU values for background and foreground and their mean (mIoU) are defined as:

$$\mathrm{IoU}_{\mathrm{bg}} = \frac{\mathrm{TN}}{\mathrm{TN} + \mathrm{FP} + \mathrm{FN}}, \qquad \mathrm{IoU}_{\mathrm{fg}} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP} + \mathrm{FN}}, \qquad \mathrm{mIoU} = \frac{\mathrm{IoU}_{\mathrm{bg}} + \mathrm{IoU}_{\mathrm{fg}}}{2} \tag{2}$$

6.2 Boundary F1-Score (BF1)

To evaluate contour alignment, a foreground pixel is defined as a boundary pixel if at least one of its 4-connected neighbors (up, down, left, right) is a background pixel, with pixels outside the 256×256 image frame treated as background. A predicted boundary pixel is considered correct if a ground-truth boundary pixel lies within its 5×5 window, and a ground-truth boundary pixel is considered found if a predicted boundary pixel lies within its 5×5 window. In `evaluate.py` and `evaluation_test.py`, both 4-neighbor boundary extraction (including image-border foreground pixels) and 5×5 window dilation are implemented via vectorized NumPy boolean array shifts. Boundary precision P, recall R, and per-image BF1 are computed as:

$$P = \frac{\text{Matched Predicted Boundary Pixels}}{\text{Total Predicted Boundary Pixels}}, \qquad R = \frac{\text{Matched True Boundary Pixels}}{\text{Total True Boundary Pixels}} \tag{3}$$

$$\mathrm{BF1} = \frac{2PR}{P + R} \tag{4}$$

and averaged across all images in the evaluation set.

6.3 Quality Score (Q) and Cost-Efficient Segmentation (CES) Score

The combined segmentation quality score Q and overall Cost-Efficient Segmentation (CES) score are given by:

$$Q = 0.70\,\mathrm{mIoU} + 0.30\,\mathrm{BF1} \tag{5}$$

$$\mathrm{CES} = \begin{cases} Q\left[1 - 0.20\,\dfrac{T_{\mathrm{CPU}}}{50\ \mathrm{ms}} - 0.10\,\min\left(1, \dfrac{S_{\mathrm{disk}}}{4\ \mathrm{MiB}}\right)\right] & \text{if } T_{\mathrm{CPU}} \le 50\ \mathrm{ms} \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

where 4 MiB = $4,194,304$ bytes.

7 Single-Threaded CPU Benchmarking Protocol

In `benchmark.py` and `predict.py`, strict single-threaded CPU execution is enforced by setting the environment variables `OMP_NUM_THREADS=1`, `MKL_NUM_THREADS=1`, `OPENBLAS_NUM_THREADS=1`, and `VECLIB_MAXIMUM_THREADS=1`, and configuring the PyTorch and OpenCV thread pools via `torch.set_num_threads(1)`, `torch.set_num_interop_threads(1)`, and `cv2.setNumThreads(1)`. At model load time (outside the timed loop), the trained `SmallUNet` checkpoint is traced and fused for CPU inference via TorchScript (`torch.jit.optimize_for_inference`), fusing sequential `Conv2d` and `BatchNorm2d` layers without altering weights or predictions. After 10 warm-up runs, each pre-resized 256×256 image in memory is timed individually using `time.process_time()`, covering BGR-to-RGB colour conversion, tensor preparation, forward inference, and binary mask conversion in memory.

8 Experimental Results

The quantitative segmentation results across the DUTS-TR validation split, the official 5,019-image DUTS-TE test set, and the six captured campus/hostel photographs are summarized in Table 3. Detailed single-core CPU timing and storage statistics on DUTS-TE are listed in Table 4.

Table 3: Overall segmentation quality and cost-efficiency metrics across all evaluation datasets.

Evaluation Split	Images	BG IoU	FG IoU	mIoU	BF1	Quality (Q)	$\mathbf{T_{CPU}}$ (ms)	$\mathbf{S_{disk}}$ (B)	CES
DUTS-TR Validation	1,056	0.85649	0.65188	0.75418	0.40062	0.64812	—	129,326	—
DUTS-TE Official Test	5,019	0.85535	0.42731	0.64133	0.30448	0.54027	25.2291	129,326	**0.48408**
Captured Photos	6	0.90394	0.53438	0.71916	0.38660	0.61939	28.12	129,326	—

Table 4: Official single-threaded CPU benchmark summary across all 5,019 DUTS-TE test images.

Benchmark Measurement	Measured Result
Total Test Images Processed (`DUTS-TE`)	5,019
Average CPU Inference Time (T_{CPU})	25.2291 ms
Median CPU Inference Time	31.2500 ms
Maximum CPU Inference Time	62.5000 ms
Saved Checkpoint Size (S_{disk})	129,326 bytes (≈ 0.1233 MiB)
Time Penalty Factor ($0.20 \times T_{\text{CPU}}/50$ ms)	0.10092
Size Penalty Factor ($0.10 \times S_{\text{disk}}/4$ MiB)	0.00308
Final Cost-Efficient Segmentation (CES) Score	**0.48408**

Substituting the official DUTS-TE test set values ($Q = 0.54027$, $T_{\text{CPU}} = 25.2291$ ms ≤ 50 ms, and $S_{\text{disk}} = 129,326$ bytes) into the CES equation yields:

$$\text{CES} = 0.54027 \times \left(1 - 0.20 \times \frac{25.2291}{50.0} - 0.10 \times \frac{129,326}{4,194,304}\right) = 0.54027 \times 0.89600 = 0.48408 \quad (7)$$

9 Personal Photograph Evaluation and Visualizations

Table 5 lists the per-image Boundary F1-scores and single-image `predict.py` CPU runtimes across all six captured photographs. For each photograph, `visualize.py` generates a four-panel comparison showing the original image, the ground-truth foreground overlay (semi-transparent red), the predicted foreground overlay (semi-transparent green), and a pixel agreement map

(where yellow indicates true-positive overlap, red indicates missed foreground pixels, and green indicates false-positive background pixels). The visual comparisons are presented in Figures 2 and 3.

Table 5: Per-image evaluation metrics across the six captured campus and hostel photographs.

Photo ID	Object	Scene & Environmental Conditions	Boundary F1	`predict.py` T_{CPU} (ms)
`photo1`	Plant	Outdoor steps, daylight, near, textured wall	0.4340	35.16
`photo2`	Bottle	Indoor desk, tube light, near, cluttered desk	0.3017	27.34
`photo3`	Shoe	Indoor floor, tube light, near, clear silhouette	0.7565	26.56
`photo4`	Occluded Packet	Indoor close-up, partial occlusion by controller	0.2869	27.34
`photo5`	Water Dispenser	Outdoor steps, daylight, far/small object	0.1570	24.22
`photo6`	Bicycle	Outdoor brick wall, daylight, thin spokes & shadow	0.3835	28.12
Aggregate	**All 6 Photos**	**Global mIoU** = 0.71916, **BG IoU** = 0.90394, **FG IoU** = 0.53438	**0.38660**	**Q = 0.61939**

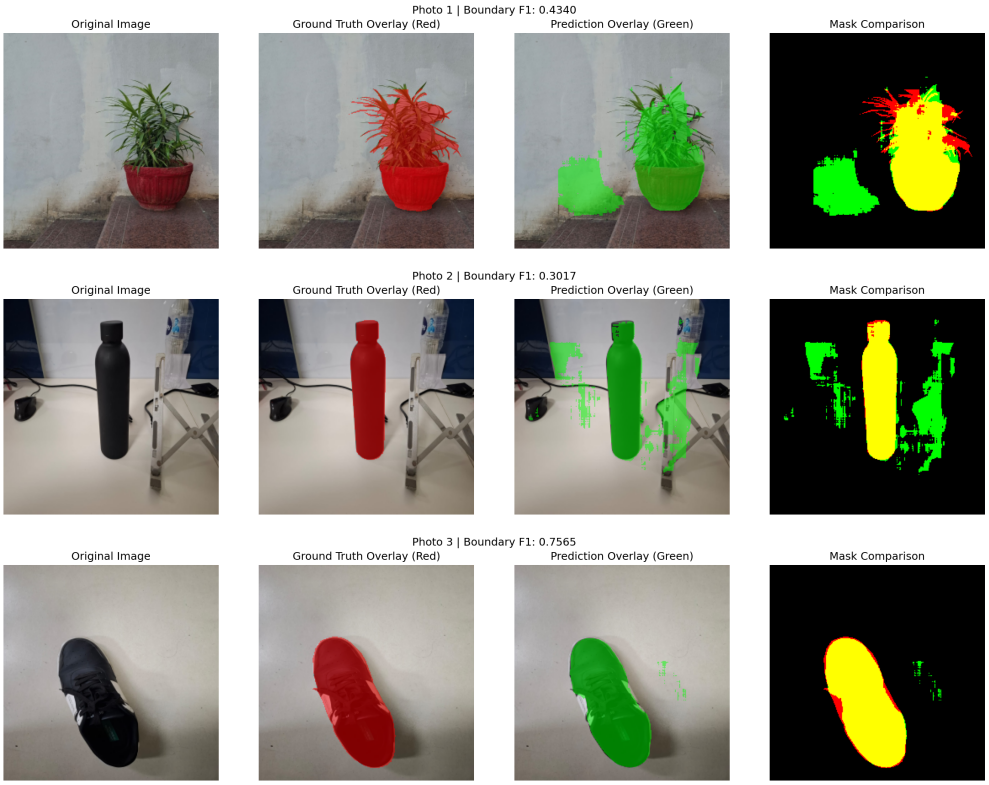


Figure 2: Four-panel qualitative segmentation comparisons for `photo1` (plant, BF1 = 0.4340), `photo2` (water bottle, BF1 = 0.3017), and `photo3` (shoe, BF1 = 0.7565).

10 Discussion and Failure Case Analysis

Successful Segmentation (`photo3` and `photo1`): The shoe photograph (`photo3`) achieved the highest boundary accuracy (BF1 = 0.7565) due to strong luminance contrast between the dark shoe and the uniform indoor floor. Similarly, the potted plant (`photo1`, BF1 = 0.4340) was segmented across the pot and main foliage, though fine leaf tips and a bright patch on the lower-left step wall introduced localized boundary errors.

Failure Case 1 — Distant Small Object and Background Clutter (`photo5`, BF1 = 0.1570): In `photo5`, the target water dispenser occupies only 1.23% of the image area at a far distance. Maintaining the full 256×256 spatial resolution improved recall on the dispenser body compared to lower-resolution models, successfully capturing the central purple cooler in yellow

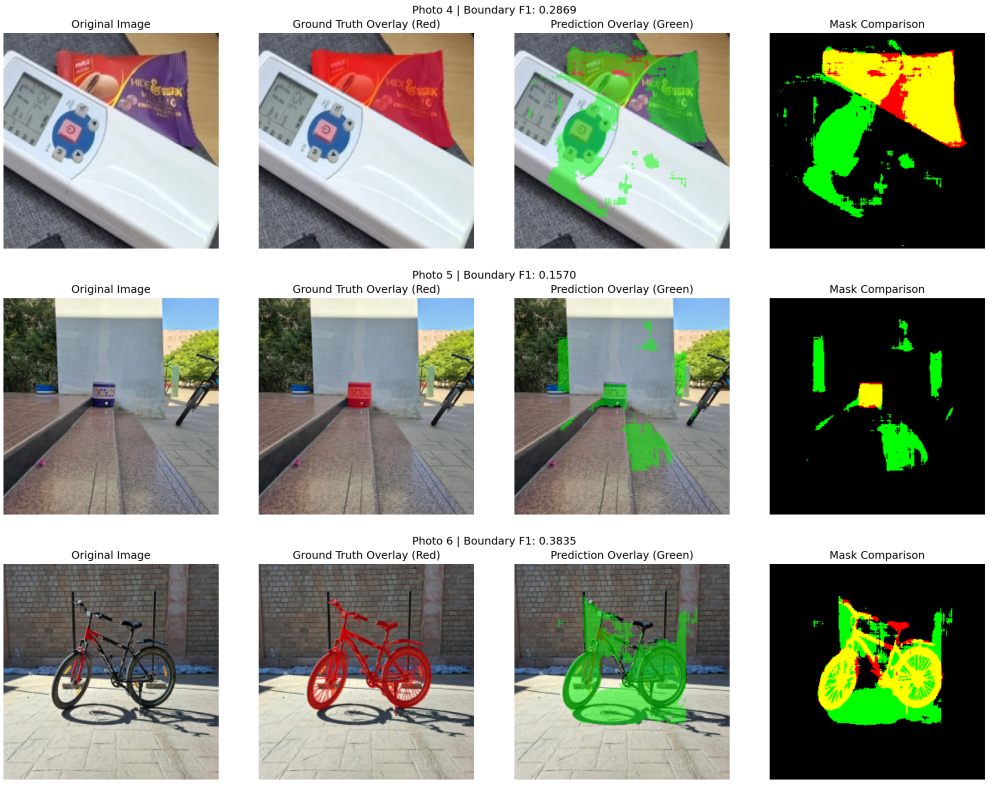


Figure 3: Four-panel qualitative segmentation comparisons for `photo4` (occluded packet/controller, BF1 = 0.2869), `photo5` (distant water dispenser, BF1 = 0.1570), and `photo6` (bicycle, BF1 = 0.3835).

on the agreement map; however, high-contrast background structures (vertical pillar edges and step reflections) still trigger false-positive green regions.

Failure Case 2 — Partial Occlusion and Multi-Component Appearance (`photo4`, BF1 = 0.2869): In `photo4`, the purple biscuit packet is partially occluded in the foreground by a bright white electronic remote/controller unit containing a high-contrast LCD and circular button pad. While the 50-epoch model successfully segments almost the entire visible biscuit packet (yellow region), it also activates on the high-contrast circular button pad of the occluding controller (green region), illustrating the challenge of partial occlusion without semantic category priors.

Failure Case 3 — Cluttered Indoor Desk and Cast Shadows (`photo2`, BF1 = 0.3017; `photo6`, BF1 = 0.3835): In `photo2`, while the entire black water bottle is recovered with almost zero false negatives, background clutter on the desk (the metallic laptop stand, background plastic bottle, and dark wall panel) induces false-positive activations. In `photo6`, the bicycle frame and wheels are captured, but the strong dark shadow cast on the sunlit pavement below the frame is misclassified as foreground.

11 Conclusion

An end-to-end ultra-lightweight binary foreground segmentation framework (`SmallUNet`) was designed, trained from scratch on DUTS-TR for 50 epochs, and evaluated under a single-core CPU budget. By combining depthwise-separable convolutions with narrow channel widths ($3 \to 8 \to 16 \to 32 \to 48$) at 256×256 resolution, the $15,949$-parameter network ($129,326$ bytes) achieves mIoU = 0.64133, BF1 = 0.30448, $Q = 0.54027$, and CES = 0.48408 on the 5,019-image DUTS-TE benchmark.